

LAPORAN PROGRAM

Membaca File Teks dan Menyimpan ke Struktur Data Vector

PENJELASAN KODE PER 5 BARIS

Baris 1-5: Include Library dan Namespace

```
cpp

#include <iostream>
#include <fstream>
#include <vector>
#include <string>
#include <cstdlib>
```

Fungsi: Mengimpor library yang diperlukan untuk program.

- `iostream`: untuk input/output standar (cout, cin)
 - `fstream`: untuk operasi file (ifstream, ofstream)
 - `vector`: untuk menggunakan struktur data vector
 - `string`: untuk manipulasi string
 - `cstdlib`: untuk fungsi system() seperti pause
-

Baris 6-10: Namespace dan Deklarasi Struct

```
cpp

using namespace std;

// Struct untuk menyimpan data dari file
struct Data {
```

Fungsi: Mendeklarasikan namespace dan membuat struktur data.

- `using namespace std`: agar tidak perlu menulis std:: setiap kali

- Komentar menjelaskan tujuan struct
 - `struct Data`: membuat tipe data baru bernama Data untuk menyimpan informasi
-

Baris 11-15: Isi Struct dan Fungsi Main

```
cpp

int id;
string nama;
int nilai;
};
```

Fungsi: Mendefinisikan anggota struct dan memulai fungsi utama.

- `int id`: menyimpan nomor ID (bilangan bulat)
 - `string nama`: menyimpan nama (string)
 - `int nilai`: menyimpan nilai (bilangan bulat)
 - Tanda kurung kurawal tutup untuk struct
-

Baris 16-20: Deklarasi Vector dan Output Judul

```
cpp

int main() {
    vector<Data> dataVector;

    cout << "=== PROGRAM BACA FILE ===" << endl << endl;
```

Fungsi: Memulai program dan membuat container untuk data.

- `int main()`: fungsi utama program dimulai
 - `vector<Data> dataVector`: membuat vector yang akan menyimpan data bertipe struct Data
 - `cout`: menampilkan judul program ke layar
 - `endl`: membuat baris baru
-

Baris 21-25: Array Nama File Alternatif

cpp

```
// Coba beberapa nama file yang mungkin
string filenames[] = {
    "bacaAku.txt",
    "bacaaku.txt",
    "BacaAku.txt",
}
```

Fungsi: Membuat array berisi variasi nama file yang mungkin.

- Komentar menjelaskan tujuan array
 - `string filenames[]`: array string untuk menyimpan nama-nama file
 - Menyimpan berbagai variasi penulisan nama file (huruf besar/kecil berbeda)
 - Ini untuk menangani kemungkinan typo atau perbedaan case-sensitive
-

Baris 26-30: Lanjutan Array dan Deklarasi Variable

cpp

```
    "BACAAKU.TXT"
};

ifstream file;
string namaFileBenar = "";
```

Fungsi: Melengkapi array dan mendeklarasikan variabel untuk file handling.

- Variasi nama file terakhir dengan huruf kapital semua
 - Kurung kurawal tutup untuk array
 - `ifstream file`: objek untuk membaca file
 - `string namaFileBenar`: menyimpan nama file yang berhasil dibuka
-

Baris 31-35: Loop Mencoba Buka File

cpp

```
// Coba satu per satu
for (int i = 0; i < 4; i++) {
    cout << "Mencoba: " << filenames[i] << " ... ";
    file.open(filenames[i]);
}
```

Fungsi: Mencoba membuka file dengan berbagai variasi nama.

- Komentar menjelaskan proses loop
 - `for (int i = 0; i < 4; i++)`: loop 4 kali sesuai jumlah variasi nama file
 - `cout`: menampilkan nama file yang sedang dicoba
 - `file.open()`: mencoba membuka file dengan nama tertentu
-

Baris 36-40: Cek File Berhasil Dibuka

```
cpp

if (file.is_open()) {
    namaFileBenar = filenames[i];
    cout << "BERHASIL!" << endl;
    break;
} else {
```

Fungsi: Mengecek apakah file berhasil dibuka.

- `if (file.is_open())`: mengecek status file apakah terbuka
 - Jika berhasil, simpan nama file ke variabel `namaFileBenar`
 - Tampilkan pesan "BERHASIL!"
 - `break`: keluar dari loop karena file sudah ditemukan
 - `else`: jika file tidak berhasil dibuka
-

Baris 41-45: Handling File Tidak Ditemukan

```
cpp
```

```

        cout << "tidak ada" << endl;
        file.clear();
    }
}

```

Fungsi: Menangani kondisi file tidak ditemukan.

- Tampilkan pesan "tidak ada"
- `file.clear()`: membersihkan status error dari stream
- Kurung kurawal tutup untuk else dan for loop
- Baris kosong untuk pemisah

Baris 46-50: Cek Apakah Ada File yang Terbuka

```

cpp

if(!file.is_open()) {
    cout << "\n[ERROR] File tidak ditemukan dengan berbagai variasi nama!" << endl;
    cout << "\nCek hal berikut:" << endl;
    cout << "1. Pastikan file bernama PERSIS: bacaAku.txt" << endl;
    cout << "2. Bukan bacaAku.txt.txt (ekstensi ganda)" << endl;
}

```

Fungsi: Memberikan pesan error jika semua percobaan gagal.

- `if(!file.is_open())`: cek apakah file masih belum terbuka
- Tampilkan pesan error dengan format yang jelas
- Berikan checklist hal-hal yang perlu dicek
- Peringatan tentang ekstensi ganda yang sering terjadi

Baris 51-55: Instruksi Lokasi dan Cara Membuat File

```

cpp

```

```
cout << "3. File di folder yang sama dengan program .exe ini" << endl;
cout << "\nCara membuat file yang benar:" << endl;
cout << "- Buka Notepad" << endl;
cout << "- Ketik data (lihat contoh di bawah)" << endl;
cout << "- File > Save As" << endl;
```

Fungsi: Memberikan instruksi detail cara membuat file.

- Instruksi tentang lokasi file yang benar
 - Panduan langkah demi langkah membuka Notepad
 - Menjelaskan cara save file dengan benar
 - Membantu user yang belum familiar dengan file handling
-

Baris 56-60: Lanjutan Instruksi Save File

cpp

```
cout << "- Save as type: pilih 'All Files (*.*)'" << endl;
cout << "- File name: bacaAku.txt" << endl;
cout << "- Klik Save" << endl;
cout << "\nContoh isi file:" << endl;
cout << "1; Tono; 82;" << endl;
```

Fungsi: Melanjutkan instruksi dan memberikan contoh isi file.

- Instruksi penting memilih "All Files" agar ekstensi benar
 - Menentukan nama file yang tepat
 - Memberikan contoh format data baris pertama
 - Format:

id; nama; nilai;

-

Baris 61-65: Contoh Data dan Exit Program

cpp

```
cout << "2; Ani; 90;" << endl;

system("pause");
return 1;
}
```

Fungsi: Menampilkan contoh data kedua dan keluar dari program.

- Contoh data baris kedua
 - `system("pause")`: menunggu user menekan tombol (Windows)
 - `return 1`: keluar dari program dengan kode error 1
 - Kurung kurawal tutup untuk blok if error
-

Baris 66-70: Pesan Sukses Buka File

cpp

```
cout << "\n[OK] File ditemukan: " << namaFileBenar << endl;
cout << "Membaca isi file...\n" << endl;

string line;
```

Fungsi: Memberikan konfirmasi file berhasil dibuka.

- Tampilkan pesan sukses dengan nama file yang ditemukan
 - Informasikan bahwa proses pembacaan dimulai
 - `string line`: variabel untuk menyimpan setiap baris yang dibaca
 - Siap untuk memulai proses pembacaan
-

Baris 71-75: Deklarasi Counter dan Mulai Loop Baca

cpp

```
int baris = 0;

while (getline(file, line)) {
    baris++;
    cout << "Baris " << baris << "; [" << line << "]" << endl;
```

Fungsi: Membaca file baris per baris dengan debugging.

- `int baris = 0`: counter untuk nomor baris
- `while (getline(file, line))`: loop membaca setiap baris sampai akhir file
- `baris++`: increment nomor baris setiap kali baca
- Tampilkan nomor baris dan isinya dalam tanda kurung siku untuk debugging

Baris 76-80: Skip Baris Kosong

cpp

```
if (line.empty() || line == "\r" || line == "\n") {
    cout << " -> Baris kosong, skip" << endl;
    continue;
}
```

Fungsi: Menangani baris kosong dalam file.

- Cek apakah baris kosong dengan beberapa kondisi
- `line.empty()`: cek string kosong
- `line == "\r"` atau `line == "\n"`: cek karakter newline
- `continue`: lanjut ke iterasi berikutnya, skip pemrosesan baris ini

Baris 81-85: Try Block dan Deklarasi Temp

cpp


```
// Parsing data
try {
    Data temp;
    size_t pos1 = line.find(';');
```

Fungsi: Memulai proses parsing dengan exception handling.

- Komentar menjelaskan mulai parsing
 - `try`: blok untuk menangkap error yang mungkin terjadi
 - `Data temp`: variabel sementara untuk menyimpan data yang diparsing
 - `size_t pos1 = line.find(';')`: mencari posisi titik koma pertama
-

Baris 86-90: Mencari Posisi Delimiter

```
cpp

size_t pos2 = line.find(',', pos1 + 1);
size_t pos3 = line.find(',', pos2 + 1);

if (pos1 == string::npos) {
    cout << " -> ERROR: Tidak ada titik koma pertama!" << endl;
```

Fungsi: Mencari semua posisi titik koma sebagai pemisah.

- `pos2`: mencari titik koma kedua mulai dari setelah `pos1`
 - `pos3`: mencari titik koma ketiga (tidak digunakan tapi dicari)
 - `string::npos`: konstanta yang berarti "tidak ditemukan"
 - Jika titik koma pertama tidak ada, tampilkan error
-

Baris 91-95: Extract ID dari String

```
cpp
```

```
        continue;
    }

    // Ambil ID
    string id_str = line.substr(0, pos1);
```

Fungsi: Mengambil substring untuk ID.

- `continue`: skip ke baris berikutnya jika error
 - Komentar menandai bagian extract ID
 - `line.substr(0, pos1)`: mengambil substring dari posisi 0 sampai sebelum pos1
 - Hasilnya string yang berisi ID dalam bentuk teks
-

Baris 96-100: Konversi ID dan Extract Nama

```
cpp

temp.id = stoi(id_str);

// Ambil Nama
string nama_str = line.substr(pos1 + 1, pos2 - pos1 - 1);
// Hapus spasi
```

Fungsi: Mengkonversi ID dan mengambil string nama.

- `stoi(id_str)`: convert string to integer, simpan ke temp.id
 - Komentar menandai bagian extract nama
 - `line.substr(pos1 + 1, pos2 - pos1 - 1)`: ambil substring antara dua titik koma
 - Komentar untuk proses selanjutnya yaitu hapus spasi
-

Baris 101-105: Pembersihan Spasi Nama (1)

```
cpp
```

```
size_t start = nama_str.find_first_not_of(" \\t\\r\\n");
size_t end = nama_str.find_last_not_of(" \\t\\r\\n");
if (start != string::npos && end != string::npos) {
    temp.nama = nama_str.substr(start, end - start + 1);
} else {
```

Fungsi: Menghapus spasi di awal dan akhir nama.

- `find_first_not_of()`: cari posisi karakter pertama yang bukan spasi/tab/newline
 - `find_last_not_of()`: cari posisi karakter terakhir yang bukan spasi/tab/newline
 - Jika kedua posisi ditemukan, ambil substring dari start sampai end
 - Ini untuk membersihkan whitespace yang tidak perlu
-

Baris 106-110: Pembersihan Spasi Nama (2) dan Extract Nilai

```
cpp

    temp.nama = nama_str;
}

// Ambil Nilai
string nilai_str = line.substr(pos2 + 1);
```

Fungsi: Finalisasi nama dan mulai extract nilai.

- Jika posisi tidak ditemukan (nama kosong), gunakan string asli
 - Kurung kurawal tutup untuk blok if-else
 - Komentar menandai mulai extract nilai
 - `line.substr(pos2 + 1)`: ambil substring setelah titik koma kedua sampai akhir
-

Baris 111-115: Pembersihan Nilai String

```
cpp
```

```
// Hapus spasi dan karakter lain
start = nilai_str.find_first_not_of(" \\t\\r\\n;");
end = nilai_str.find_last_not_of(" \\t\\r\\n;");
if (start != string::npos && end != string::npos) {
    nilai_str = nilai_str.substr(start, end - start + 1);
}
```

Fungsi: Membersihkan string nilai dari karakter tidak perlu.

- Komentar menjelaskan proses pembersihan
 - Cari karakter pertama dan terakhir yang bukan spasi, tab, newline, atau titik koma
 - Jika ditemukan, ambil substring yang bersih
 - Ini penting karena nilai bisa diikuti titik koma dan spasi
-

Baris 116-120: Konversi Nilai dan Push ke Vector

```
cpp
    }
    temp.nilai = stoi(nilai_str);

    dataVector.push_back(temp);
    cout << " -> Berhasil: ID=" << temp.id << " Nama=" << temp.nama << " Nilai=" << temp.nilai << endl;
```

Fungsi: Menyimpan nilai dan menambahkan ke vector.

- Kurung kurawal tutup untuk if pembersihan nilai
 - `stoi(nilai_str)`: convert string nilai ke integer
 - `dataVector.push_back(temp)`: menambahkan struct temp ke akhir vector
 - Tampilkan konfirmasi data berhasil diparsing dengan detail lengkap
-

Baris 121-125: Catch Exception

```
cpp
```

```

    } catch (exception& e) {
        cout << " -> ERROR parsing: " << e.what() << endl;
    }
}

```

Fungsi: Menangkap dan menampilkan error parsing.

- Kurung kurawal tutup untuk try block
 - `catch (exception& e)`: menangkap semua jenis exception
 - `e.what()`: mendapatkan pesan error dari exception
 - Tampilkan pesan error jika terjadi kesalahan (misal konversi gagal)
 - Kurung kurawal tutup untuk while loop
-

Baris 126-130: Tutup File dan Header Output

```

cpp

file.close();

// Tampilkan hasil akhir
cout << "\n===== " << endl;

```

Fungsi: Menutup file dan memulai tampilan hasil.

- Baris kosong untuk pemisah
 - `file.close()`: menutup file yang sudah selesai dibaca
 - Komentar menandai bagian output hasil
 - Tampilkan garis pembatas untuk memisahkan proses dan hasil
-

Baris 131-135: Judul dan Jumlah Data

```

cpp

```

```

cout << "HASIL AKHIR" << endl;
cout << "===== " << endl;
cout << "Total data berhasil dibaca: " << dataVector.size() << endl << endl;

if (dataVector.size() > 0) {

```

Fungsi: Menampilkan header hasil dan jumlah data.

- Tampilkan judul "HASIL AKHIR"
- Garis pembatas bawah
- `dataVector.size()`: menampilkan jumlah elemen dalam vector
- Cek apakah ada data yang berhasil dibaca

Baris 136-140: Loop Menampilkan Data

cpp

```

for (size_t i = 0; i < dataVector.size(); i++) {
    cout << "Data ke-" << (i+1) << ":" << endl;
    cout << " ID   : " << dataVector[i].id << endl;
    cout << " Nama : " << dataVector[i].nama << endl;
    cout << " Nilai : " << dataVector[i].nilai << endl;

```

Fungsi: Menampilkan setiap data dalam vector.

- Loop dari 0 sampai ukuran vector
- `(i+1)`: menampilkan nomor data mulai dari 1 (bukan 0)
- `dataVector[i].id`: akses field id dari elemen ke-i
- `dataVector[i].nama`: akses field nama dari elemen ke-i
- `dataVector[i].nilai`: akses field nilai dari elemen ke-i

Baris 141-145: Garis Pembatas dan Else

cpp

```

        cout << "-----" << endl;
    }
} else {
    cout << "Tidak ada data yang berhasil dibaca!" << endl;
}

```

Fungsi: Menyelesaikan tampilan data atau pesan kosong.

- Tampilkan garis pembatas setelah setiap data
- Kurung kurawal tutup untuk for loop
- `else`: jika vector kosong (size = 0)
- Tampilkan pesan bahwa tidak ada data
- Kurung kurawal tutup untuk blok if-else

Baris 146-150: Pause dan Return

```

cpp

system("pause");
return 0;
}

```

Fungsi: Menunggu input user dan mengakhiri program.

- Baris kosong
- `system("pause")`: program menunggu user tekan tombol (Windows)
- `return 0`: keluar dari fungsi main dengan kode sukses (0 = tidak ada error)
- Kurung kurawal tutup untuk fungsi main
- Program selesai

KESIMPULAN

Program ini berhasil membaca file teks dengan format terstruktur dan menyimpannya ke dalam vector. Fitur-fitur utama:

1. **Multi-attempt file opening:** Mencoba berbagai variasi nama file

2. **Robust parsing:** Menangani spasi dan karakter tidak perlu
3. **Error handling:** Try-catch untuk menangkap error konversi
4. **Detailed debugging:** Menampilkan proses pembacaan setiap baris
5. **User-friendly output:** Pesan error yang jelas dan instruksi lengkap

Program ini cocok untuk pembelajaran file handling, struct, vector, dan string manipulation dalam C++.